Task: Implement a browser-based 3D physics platformer featuring a futuristic car that drives forward through an endless, procedurally generated obstacle course. The car can drive on floors, walls, and ceilings, use limited fuel-powered thrusters to fly, rotate around its longitudinal axis (barrel rolls), and activate suction to attach to surfaces.

The objective is to travel as far forward as possible without falling or crashing, using a combination of driving, controlled flight, rotation, and suction.

Context:

- New project using TypeScript, Three.js, Rapier physics, React, and Vite.
- Inspired by Geometry Dash, Doodle Jump, Trackmania, and Distance.
- Unlike Doodle Jump, progression is **horizontal and forward through a fully 3D environment**, not vertically upward.
- The world consists of connected and disconnected sections of floors, ceilings, left walls, and right walls.
- All four surfaces can appear and disappear independently as the player progresses.
- The game is played from a third-person perspective, with a camera following the car.

Requirements:

1. Core movement and controls

Implement a physics-driven car with the following abilities:

Driving

- The car moves forward along the surface it is attached to or resting on.
- Allow acceleration and braking.
- Support driving on floors, ceilings, left walls, and right walls.
- Maintain consistent forward movement regardless of the surface orientation.
- Preserve momentum when leaving a surface.
- Driving should feel responsive and arcade-like rather than requiring a realistic vehicle simulation.

Flight and thrusters

- Equip the car with limited fuel-powered thrusters.
- Thrusters allow the player to leave a surface, cross gaps, and travel toward another surface.
- Thrust is controlled independently of regular driving.
- The car must be able to cross from a floor to a ceiling, from a wall to another wall, or between any other combination of surfaces.
- Fuel is consumed while thrusters are active.
- When fuel reaches zero, thrust becomes unavailable until fuel is replenished.

Rotation

- Allow rotation around the car's forward-facing longitudinal axis.
- Use the left and right arrow keys for barrel-roll rotation.
- Support continuous rotation, including complete 360-degree rolls.
- Do not implement front flips, back flips, pitch control, or free yaw rotation in the initial version.
- Rotation should allow the player to orient the car's wheels toward a floor, wall, or ceiling before attempting to attach.

Truster acceleration

- Allow to accelerate forwards with trustees. Same fuel consumption as other trusters

Suction

- Implement a manually activated suction mechanism.
- Assign a dedicated key, such as `E`, to engage or disengage suction.
- Suction is disabled by default.
- The car must not automatically stick to walls or ceilings without suction being activated.
- When suction is active and the wheels face a nearby valid surface, apply an attraction force toward that surface.
- Require the car to be sufficiently close and correctly oriented to attach.
- Once attached, allow the car to drive along the surface.
- Disengaging suction releases the car from the attached surface.
- Suction must not teleport or instantly snap the car across large distances.
- Prevent attachment when a surface is too far away or the wheels are facing the wrong direction.
- Normal ground contact should allow driving without suction, while inverted and vertical driving require active suction.
- No matter of the perspective of the car the turning controls should stay the same (eg. if the car is on the left, turning left would move car up the wall from perspective of player)

2. Controls

Implement keyboard controls with the following initial bindings:

Key	Action
`W`	Accelerate forward
`S`	Brake / reverse
`Space`	Activate main thrusters
`←`	Rotate car left (barrel roll)

`→`	Rotate car right (barrel roll)
`E`	Toggle suction on/off
`R`	Restart attempt
`Esc`	Pause game
`Arrow up`	Forward truster

- Make all controls responsive and support simultaneous key presses.
- Allow acceleration, thrust, rotation, and suction to be used in combination.
- Display the control scheme in a simple in-game help overlay.
- Prevent browser default actions from interfering with gameplay.

3. Fully 3D procedural environment

Create an endless obstacle course that extends along a forward axis.

The environment must contain four independently generated surface types:

- **Floor:** Horizontal platforms beneath the player.
- **Ceiling:** Horizontal platforms above the player.
- **Left wall:** Vertical surfaces to the player's left.
- **Right wall:** Vertical surfaces to the player's right.

Each surface can start, end, disappear, or reappear independently.

For example:

- A section may contain only a floor.
- The next section may contain only a ceiling.
- Another section may contain a right wall and ceiling.
- Another may contain both side walls but no floor.
- Some sections may contain all four surfaces.
- Some sections may require a flight transition because no continuous drivable surface exists.

Procedural generation rules:

- Generate new sections ahead of the player.
- Remove sections far behind the player to maintain performance.
- Randomize surface availability, length, gaps, and transitions.
- Gradually increase difficulty as the player travels farther.
- Keep the corridor's overall dimensions consistent initially.

- Avoid generating impossible sections that cannot be completed with the available movement capabilities and fuel.
- Ensure at least one achievable route through generated sections.
- Support seeded procedural generation for repeatable runs.
- Avoid placing geometry directly inside the moving car.

4. Example gameplay scenario

Implement an initial playable sequence demonstrating the core mechanics.

Section 1 — Normal driving

- The player starts on a flat floor.
- No suction or thrusters are required.
- The player accelerates forward normally.

Section 2 — Floor disappears, ceiling appears

- The approaching floor ends.
- A ceiling platform begins farther ahead.
- The player must use the thrusters to leave the floor and reach the ceiling.
- While airborne, the player uses the left or right arrow key to rotate the car upside down.
- The player activates suction using `E`.
- If the wheels are facing the ceiling and the car gets sufficiently close, suction pulls the vehicle toward it.
- Once attached, the player continues driving forward along the ceiling.

Section 3 — Transition to a side wall

- The ceiling eventually ends.
- A left or right wall appears.
- The player uses thrust and a 90-degree barrel roll to orient the wheels toward the wall.
- Suction allows the vehicle to attach to the wall.
- The player continues driving forward along the wall.

Section 4 — Combined obstacles

- Generate increasingly complicated sequences combining floors, ceilings, walls, and gaps.
- Require strategic fuel usage, accurate rotation, and correctly timed suction activation.
- The player continues until they fall, crash, or become unable to progress.

5. Physics and surface attachment

Use Rapier for rigid-body physics and collision detection.

- Implement consistent forward acceleration across differently oriented surfaces.

- Use gravity to affect the vehicle whenever it is unsupported.
- Apply custom suction forces for wall and ceiling attachment.
- Handle transitions between normal contact, flight, and suction attachment.
- Preserve linear and angular momentum when leaving a surface.
- Avoid violent snapping, unstable collisions, and uncontrolled spinning when attaching.
- Detect valid attachment surfaces using raycasts or suitable collision queries.
- Allow the car to rotate freely around its longitudinal axis while airborne.
- Stabilize the car appropriately while attached without preventing intentional detachment.
- Use a fixed physics timestep independent of rendering FPS.
- Prioritize predictable arcade-style handling over realistic car suspension.

6. Fuel management

- Give the vehicle a tank supporting approximately five seconds of continuous thrust.
- Consume fuel proportionally to thruster usage.
- Introduce fuel recharge pickups or stations along the course.
- Make recharge locations clearly visible.
- Restore fuel when the car collects a pickup or reaches a station.
- Prevent thruster activation when the tank is empty.
- Make fuel placement part of the procedural generation rules.
- Ensure the player can reach future fuel pickups by combining driving, suction, momentum, and limited flight.
- Suction does not consume thruster fuel in the initial version.

7. Camera

Implement a third-person camera following behind the car.

- Maintain a clear view of upcoming obstacles.
- Keep the camera oriented primarily along the world's forward direction.
- Do not rotate the entire camera when the car performs barrel rolls.
- Allow the car to rotate inside the camera view while keeping floors, ceilings, and walls visually understandable.
- Smoothly follow the vehicle's position and changes in height.
- Adjust the camera to accommodate the car driving on side walls and ceilings.
- Prevent the camera from clipping through geometry where practical.
- Prioritize visibility and spatial awareness over cinematic camera movement.

8. Game progression and failure

- Track total forward distance as the primary score.
- Display current distance, fuel remaining, and suction status.
- End the attempt when the car falls sufficiently far outside the playable corridor, collides with fatal obstacles, or otherwise reaches an unrecoverable state.
- Do not treat ordinary contact with drivable surfaces as a crash.
- Allow immediate restart using `R`.

- Reset the vehicle, physics state, camera, procedural environment, and fuel on restart.
- Gradually increase difficulty as the player's distance increases.
- Make the game challenging through combinations of movement mechanics rather than arbitrary unavoidable obstacles.

9. Visual presentation

Create a clean, futuristic, stylized 3D environment.

- Use simple but polished geometry.
- Represent the car with a recognizable futuristic vehicle model or a custom low-poly approximation.
- Visually distinguish floors, ceilings, and side walls.
- Add clear visual effects for thruster activation.
- Indicate when suction is enabled and when a surface attachment succeeds.
- Add subtle visual feedback for refuelling.
- Ensure gaps and transitions are clearly visible from the camera perspective.
- Use lighting, depth cues, and contrasting materials to make the 3D environment readable.
- Implement a minimal HUD with distance, fuel, and suction indicators.
- Prioritize responsive gameplay and visual clarity over photorealism.

10. Technical constraints

- Use TypeScript, Three.js, Rapier, React, and Vite.
- Keep game physics, procedural generation, rendering, and input handling modular.
- Use deterministic fixed-timestep physics updates.
- Optimize procedural geometry creation and cleanup.
- Maintain smooth performance in modern desktop browsers.
- Use keyboard controls initially.
- Avoid unnecessary dependencies.
- Do not implement multiplayer, online leaderboards, accounts, or cloud infrastructure.
- Do not implement RL, AI agents, training APIs, Gymnasium integration, reward functions, or research-specific features.
- Do not use Tesla branding or proprietary vehicle models.

Acceptance criteria:

1. The project launches locally and renders a fully 3D playable environment.
2. The car can drive forward along a floor using `W`.
3. Holding `Space` activates thrusters and consumes fuel.
4. The player can perform full barrel rolls using the left and right arrow keys.
5. Front flips, back flips, and unrestricted 3D rotation are not possible.
6. Pressing `E` toggles suction.
7. The car can attach to a ceiling only when suction is active, the wheels face the ceiling, and the car is close enough.
8. The car can similarly attach to left and right walls.

9. The player can drive forward while attached to any of the four surface types.
10. The player can successfully complete the initial floor-to-ceiling transition using thrust, a 180-degree barrel roll, and suction.
11. The player can transition from a ceiling to a side wall using thrust, rotation, and suction.
12. Floors, ceilings, left walls, and right walls appear and disappear independently during procedural generation.
13. The course extends forward indefinitely, with new sections generated ahead of the player.
14. Fuel runs out after approximately five seconds of continuous thrust and can be replenished through pickups.
15. The camera follows the car without rotating with barrel rolls.
16. Distance, fuel, and suction status are displayed correctly.
17. Falling outside the playable environment triggers game over.
18. Restarting correctly resets the game.
19. Automated tests cover fuel consumption, suction attachment conditions, movement transitions, seeded